

Vulnerability Assessment Report

Generated on 2026-06-08 11:03 UTC

Total vulnerabilities	11
Critical	0
High	2
Medium	4
Low	1
Info	4

Executive Summary

This report lists unique findings from the security scan, grouped by severity. Address critical and high severity issues first.

Severity	Count	Description
Critical	0	Immediate action required.
High	2	Fix as soon as possible.
Medium	4	Plan remediation.
Low	1	Consider in next release.
Info	4	Informational only.

Findings

1. Content Security Policy (CSP) Header Not Set

MEDIUM

CWE: 693

Description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

URL	http://test-victim:9999/
Method	GET

Recommendation

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Tags

[OWASP_2021_A05](#)

[SYSTEMIC](#)

[CWE-693](#)

[OWASP_2017_A06](#)

[OWASP_2025_A02](#)

2. Missing Anti-clickjacking Header

MEDIUM

CWE: 1021

Description

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

URL	http://test-victim:9999/robots.txt
Method	GET

Recommendation

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Tags

[OWASP_2021_A05](#)

[CWE-1021](#)

[SYSTEMIC](#)

[WSTG-v42-CLNT-09](#)

[OWASP_2017_A06](#)

[OWASP_2025_A02](#)

3. Application Error Disclosure

MEDIUM

CWE: 550

Description

This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.

URL	http://test-victim:9999/robots.txt
Method	GET

Recommendation

Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server

side and not exposing them to the user.

Tags

[WSTG-v42-ERRH-02](#)

[WSTG-v42-ERRH-01](#)

[OWASP_2021_A05](#)

[CWE-550](#)

[OWASP_2017_A06](#)

[OWASP_2025_A02](#)

4. Absence of Anti-CSRF Tokens

MEDIUM

CWE: 352

Description

No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including:

- * The victim has an active session on the target site.
- * The victim is authenticated via HTTP auth on the target site.
- * The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

URL	http://test-victim:9999/mini3/guestbook
Method	GET

Recommendation

Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change. Phase: Implementation Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Tags

[OWASP_2021_A01](#)
[SYSTEMIC](#)
[WSTG-v42-SESS-05](#)
[OWASP_2025_A01](#)
[OWASP_2017_A05](#)
[CWE-352](#)

5. Information Disclosure - Sensitive Information in URL

INFORMATIONAL

CWE: 598

Description

The request appeared to contain sensitive information leaked in the URL. This can violate PCI and most organizational compliance policies. You can configure the list of strings for this check to add or remove values specific to your environment.

URL	http://test-victim:9999/mini11/welcome?user=guest
Method	GET

Recommendation

Do not pass sensitive information in URIs.

Tags

[CWE-598](#)
[OWASP_2021_A01](#)
[SYSTEMIC](#)
[OWASP_2017_A03](#)
[OWASP_2025_A01](#)

6. User Agent Fuzzer

INFORMATIONAL

CWE: 0

Description

Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.

URL	http://test-victim:9999/mini12/feedback
-----	---

Method	POST
--------	------

Tags

SYSTEMIC

7. Server Leaks Version Information via "Server" HTTP Response Header Field

LOW

CWE: 497

Description

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

URL	http://test-victim:9999/robots.txt
Method	GET

Recommendation

Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Tags

OWASP_2021_A05

SYSTEMIC

OWASP_2017_A06

WSTG-v42-INFO-02

CWE-497

OWASP_2025_A02

8. Authentication Request Identified

INFORMATIONAL

CWE: -1

Description

The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.

URL	http://test-victim:9999/mini5/login
-----	-------------------------------------

Method	POST
---------------	------

Recommendation

This is an informational alert rather than a vulnerability and so there is nothing to fix.

9. SQL Injection - SQLite

HIGH

CWE: 89

Description

SQL injection may be possible.

URL	http://test-victim:9999/mini5/login
Method	POST

Recommendation

Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do **not** concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.

Tags

[OWASP_2021_A03](#)

[PCI_DSS](#)

[CWE-89](#)

[OWASP_2025_A05](#)

[WSTG-v42-INPV-05](#)

[HIPAA](#)

[OWASP_2017_A01](#)

[API_2023_API10](#)

10. User Controllable HTML Element Attribute (Potential XSS)

INFORMATIONAL

CWE: 20

Description

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

URL	http://test-victim:9999/mini2/profile?name=test
Method	GET

Recommendation

Validate all input and sanitize output it before writing to any HTML attributes.

Tags

[OWASP_2025_A05](#)

[OWASP_2021_A03](#)

[CWE-20](#)

[OWASP_2017_A01](#)

11. Cross Site Scripting (Reflected)

HIGH

CWE: 79

Description

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology. When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise. There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based. Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash. Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

URL	http://test-victim:9999/mini3/guestbook
Method	POST

Recommendation

Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies. For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters. Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed. Phase: Architecture and Design For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Phase: Implementation For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping. To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Tags

[CWE-79](#)

[OWASP_2021_A03](#)

[PCI_DSS](#)

[WSTG-v42-INPV-01](#)

[OWASP_2025_A05](#)

[HIPAA](#)

[OWASP_2017_A07](#)